

Quantifying the Blast Radius: Centrality-Based Criticality Modeling for Infrastructure Risk — Project Instructions

1. Project Overview

- **Short Description:** Graph-theoretic blast radius and criticality analysis of a synthetic enterprise infrastructure topology, using betweenness, eigenvector, and PageRank centrality to identify which nodes' failure or compromise would cause the most operational damage.
- **Description:** This project models a fintech infrastructure estate — 90 nodes spanning edge services, applications, data stores, compute infrastructure, external dependencies, core business systems, and operational tooling — as a directed dependency graph. Three centrality metrics (betweenness, eigenvector, PageRank) are computed independently and compared to identify critical nodes from complementary structural perspectives. Node-removal simulation quantifies blast radius directly: for each candidate critical node, how many downstream nodes become unreachable, and which customer-facing or revenue-generating paths break. A business impact layer translates structural criticality into operational language, producing a remediation backlog ranked by blast radius weighted by business consequence.
- **Abstract:** Centrality scores are easy to compute and easy to misread. A node with high betweenness centrality is mathematically a chokepoint, but without a topology specific enough to make that chokepoint legible — what the node does, what depends on it, what breaks without it — the finding remains abstract. This project builds a 90-node synthetic infrastructure topology for a fintech estate and applies three independent centrality measures (betweenness, eigenvector, PageRank) to identify structurally critical nodes, then validates each candidate through direct node-removal simulation rather than relying on the centrality score alone. The three metrics are deliberately compared rather than averaged: where they agree, the finding is robust; where they diverge, the divergence itself indicates that a node is critical for different structural reasons, which changes the remediation approach. A business impact layer maps each node to estimated revenue exposure and customer-facing consequence, converting a graph-theoretic ranking into a board-legible prioritization. The analysis surfaces an authentication gateway with no redundant path serving as a structural single point of failure for 60% of revenue-generating traffic — a finding invisible to conventional infrastructure inventories but immediate once the dependency graph is traversed.
- **Target Audience:** CISOs, infrastructure and platform engineering leadership, enterprise risk officers, board-level risk committees
- **Output Format:** HTML (self-contained)
- **Render Command:** `quarto render index.qmd`
- **Expected Output Path:** `output/index.html`
- **Data Sources:**
 - `data/nodes.csv` — 90 infrastructure nodes with tier, business function, and revenue-impact attributes
 - `data/edges.csv` — directed dependency edges (A depends on B)
 - `data/centrality_scores.csv` — pre-computed betweenness, eigenvector, and PageRank scores
 - `data/blast_radius.csv` — pre-computed node-removal simulation results
 - Generated synthetically via `generate_infra_data.py`, `random.seed(42)`
- **Project Status:** In Progress

2. Directory Structure

```
nexacore_blast_radius/
├── data/                    # Synthetic infrastructure topology (CSV)
│   ├── nodes.csv
│   ├── edges.csv
│   ├── centrality_scores.csv
│   └── blast_radius.csv
├── scripts/
│   ├── generate_infra_data.py # Synthetic topology generator
│   ├── compute_centrality.py  # Pre-computes all centrality + blast radius metrics
│   └── pipeline_diagnostic.R   # Out-of-Quarto pipeline verification
├── output/                  # Rendered HTML output
├── _brand.yml
├── _quarto.yml
├── INSTRUCTIONS.md          # This file
└── index.qmd                # Main Quarto entry point
```

3. Document Sections — Required & Ordered

3.1 Setup — Libraries, brand colors, theme_brand() definition, data ingestion 3.2 Introduction — Problem statement: why centrality scores feel abstract without topology; relationship to attack-path analysis (complementary lens: adversarial reachability vs. structural/operational failure) 3.3 Graph Theory Recap: Centrality — Short conceptual section (300–400 words): betweenness, eigenvector, PageRank explained in plain language, why three metrics rather than one, what agreement and divergence each mean 3.4 The Infrastructure Topology — Descriptive overview: 90 nodes across 7 tiers, dependency edge count, tier-level summary tables 3.5 Graph Construction — Building the directed dependency graph

with `igraph/tidygraph`; graph summary statistics 3.6 Centrality Analysis — Three metrics computed and ranked independently; comparison table showing agreement/divergence across top candidates; visualization of the full graph colored by composite criticality 3.7 Blast Radius Validation — Node-removal simulation for top centrality candidates; downstream unreachability counts; validates centrality findings against direct structural consequence 3.8 Business Impact Translation — Mapping structural blast radius to revenue exposure and customer-facing consequence; composite criticality score (structural x business weight) 3.9 Remediation Prioritization — Ranked backlog: composite criticality, remediation type (add redundancy / re-architect / monitor), estimated effort 3.10 Key Takeaways & Conclusion — 2–3 insights (300–400 words) + conclusion (150–200 words) 3.11 Session Information — `sessioninfo::session_info()` chunk, `echo: false` 3.12 Footer — Rendered with Quarto + package attribution line

4. YAML Header

```
---
title: "Quantifying the Blast Radius: Centrality-Based Criticality Modeling for Infrastructure Risk"
subtitle: "A Structural Risk Case Study – NexaCore Financial Technologies"
author: "Patrick Lefler"
abstract: |
  Centrality scores are easy to compute and easy to misread. A node with high betweenness
  centrality is mathematically a chokepoint, but without a topology specific enough to make
  that chokepoint legible, the finding remains abstract. This project builds a 90-node
  synthetic infrastructure topology for a fintech estate and applies three independent
  centrality measures – betweenness, eigenvector, and PageRank – to identify structurally
  critical nodes, then validates each candidate through direct node-removal simulation rather
  than relying on the centrality score alone. The three metrics are deliberately compared
  rather than averaged: where they agree, the finding is robust; where they diverge, the
  divergence itself indicates that a node is critical for different structural reasons. A
  business impact layer maps each node to estimated revenue exposure and customer-facing
  consequence, converting a graph-theoretic ranking into a board-legible prioritization. The
  analysis surfaces an authentication gateway with no redundant path serving as a structural
  single point of failure for a majority of revenue-generating traffic – a finding invisible
  to conventional infrastructure inventories but immediate once the dependency graph is
  traversed.
date: "YYYY-MM-DD"
format:
  html:
    code-fold: true
    code-copy: true
    code-overflow: wrap
    code-tools: false
    code-summary: "Display code"
    df-print: kable
    embed-math: true
    embed-resources: true
    fig-align: center
    fig-height: 6
    fig-width: 10
    highlight-style: arrow
    lightbox: true
    linkcolor: "#0166CC"
    number-sections: false
    page-layout: full
    smooth-scroll: true
    theme: sandstone
    toc: true
    toc-depth: 3
    toc-location: right
    toc-title: "Contents"
execute:
  echo: true
  warning: false
  message: false
html-math-method: mathjax
knitr:
  opts_chunk:
    comment: "#>"
---
```

5. Brand & Theme Configuration

`_brand.yml` identical to the AD Identity Governance project — confirm present in root.

```
brand_primary <- "#1A1A2E"
brand_secondary <- "#16213E"
brand_accent <- "#0F3460"
brand_highlight <- "#E94560"
brand_surface <- "#F5F5F5"
brand_text <- "#1A1A2E"
```

```
brand_palette <- c(
  primary   = brand_primary,
  secondary = brand_secondary,
  accent    = brand_accent,
  highlight = brand_highlight
)
```

Tier-based node coloring for graph visualizations:

```
tier_colors <- c(
  "Edge / Perimeter"      = "#4A90D9",
  "Application / Service" = brand_secondary,
  "Data"                  = brand_accent,
  "Infrastructure"        = "#8899AA",
  "External Dependency"   = "#E8A838",
  "Core Business Systems" = brand_highlight,
  "Monitoring / Ops"      = "#4C7A6B"
)
```

Metric-specific color mapping (for centrality comparison visuals):

```
metric_colors <- c(
  "Betweenness" = brand_accent,
  "Eigenvector" = brand_secondary,
  "PageRank"    = brand_highlight
)
```

6. Visualization Rules

- Default stack: `ggraph` (graph layouts) → `ggplotly()` → `plotly` direct
- `ggraph` with `layout = "fr"` (Fruchterman-Reingold) for full topology views; `layout = "stress"` as an alternative if `fr` produces excessive overlap at 90 nodes
- `scale_fill_manual` / `scale_color_manual` using `tier_colors` and `metric_colors`
- Node size in full-topology visualizations scaled to composite criticality score — visually reinforces the ranking without requiring the reader to cross-reference a table
- Centrality comparison: small-multiples or paired bar chart, never a single blended score presented as if it were one metric

7. Table Rules

- Default: `kable` + `kableExtra` → `gt` → `DT::datatable()`
- Default kable setup identical to AD project:

```
kable(
  data,
  format      = "html",
  digits      = 3,
  caption     = "Table N: [Description]",
  col.names   = c("Col 1", "Col 2", "Col 3")
) |>
kable_styling(
  bootstrap_options = c("striped", "hover", "condensed"),
  full_width        = TRUE,
  position          = "left",
  font_size         = 13
)
```

- **Lesson carried forward from the AD project's render failures:** do not use `column_spec()` on any table whose source data passed through `notes`-style free-text fields without explicit ASCII sanitization via `iconv(..., from = "UTF-8", to = "ASCII", sub = "-")`. Default to `kable_styling()` alone; add `column_spec()` only on tables built from clean, fully-controlled tibbles with no free-text columns.
- `DT::datatable()` for the full 90-node centrality comparison table where interactive sorting adds value

8. R Libraries

```
library(kableExtra) # Table formatting
library(knitr)      # Document rendering
library(plotly)     # Interactive chart wrapping
library(scales)     # Axis and label formatting
library(sessioninfo) # Session provenance
library(tidyverse)  # Data manipulation and ggplot2
```

```
# Project-specific
```

```
library(igraph)      # Core graph construction, centrality, node-removal simulation
library(tidygraph)   # Tidy API for igraph objects
library(ggraph)      # Graph visualization (ggplot2 extension)
library(DT)          # Interactive datatable for full node/centrality exploration
```

9. Writing Standards

- Voice: Third person, direct, precise
- Grammar: Vary sentence length deliberately. Avoid flat, homogeneous AI-generated rhythm.
- Structural Rules: Lead with the non-obvious insight, not the methodology. One idea per paragraph. No bullet-point dumps masquerading as analysis. Lists for genuinely enumerable items only. Minimize em-dashes — and per the AD project's render lessons, **avoid em-dashes and en-dashes inside any data field that will pass through `kable()` / `kableExtra`; ASCII hyphens only in CSV-sourced text columns.**
- Banned vocabulary: delve, leverage (verb), harness, unlock, seamlessly, robust (outside statistical context), transformative, elevate, navigate (metaphor), landscape (metaphor), ecosystem (metaphor), paradigm, game-changer, cutting-edge, state-of-the-art, empower, innovative (unless citing a specific novel technique), holistic, synergy, streamline, deep dive (noun), unpack (metaphor), "it's worth noting", "it is important to note", "in today's world", "in conclusion" (never opens a closing paragraph)
- Domain-specific terms to use consistently:
 - **Node** — preferred over "asset" or "component" in graph-analytical sections; "service," "system," or the node's specific name in business-impact prose
 - **Blast radius** — the count of downstream nodes/paths affected by a given node's failure or compromise; distinct from "centrality," which is a structural property independent of simulation
 - **Critical node** — a node identified as high-priority by at least one centrality metric AND confirmed by blast radius simulation; a node flagged by centrality alone but not confirmed by simulation is a "candidate," not yet "critical"
 - **Composite criticality score** — the business-weighted combination of structural blast radius and revenue/customer impact; always distinguished from raw centrality scores in prose
 - **Single point of failure (SPOF)** — reserved for nodes with no redundant path; used precisely, not as a general intensifier
- Number formatting: Percentages: one decimal place (99.2%). Monetary values: \$X,XXX format. Statistical intervals: bracket notation [lower, upper].
- Figures and tables carry their own narrative weight — captions add information, not repeat axis labels.

10. Deliverables Checklist

- ☐ `index.qmd` — primary rendered document
- ☐ `_brand.yml` — confirmed in root
- ☐ `README.md` — complete
- ☐ Abstract — embedded in YAML, ~200 words
- ☐ Output HTML — confirmed self-contained (`embed-resources: true`)
- ☐ LinkedIn post (if requested)

11. README.md Structure

Same template as the AD Identity Governance project (see that project's README.md for the full structure). Key Findings section should lead with the authentication gateway SPOF finding and the centrality-divergence finding (which nodes are flagged by some metrics but not others, and what that implies for remediation).

12. Open Issues & Decisions Log

- [2026-06-26] — Confirmed: directed dependency graph (A depends on B, not symmetric)
- [2026-06-26] — Confirmed: 90-node topology across 7 tiers. Considered increasing to 130 nodes for "realism"; decided against it — realism comes from topology design quality (fan-out ratios, redundancy patterns), not node count. Marginal nodes beyond 90 would add narrative/design burden without corresponding analytical payload, and risk a "hairball" full-topology visualization that undermines board legibility. Effort redirected to fan-out and structural risk design below.
- [2026-06-26] — Confirmed: standalone narrative, not an explicit extension of the AD project's NexaCore environment (same fictional company, but the infrastructure topology is introduced fresh with its own context rather than assuming familiarity with the AD project)
- [2026-06-26] — Centrality computation approach: pre-compute betweenness, eigenvector, and PageRank in `compute_centrality.py` (or equivalent R script) and write to `data/centrality_scores.csv`, following the AD project's lesson that complex graph computation inside Quarto chunks is fragile to render — read pre-computed results in the document

- [2026-06-26] — Blast radius computation: pre-compute node-removal simulation results to `data/blast_radius.csv` for the same reason; do not attempt live node-removal loops inside Quarto chunks
- [2026-06-26] — Layout selection for 90-node full-topology visualization: `fr` vs. `stress` layout in `ggraph` to be evaluated once the topology is generated; `fr` may produce excessive overlap at this node count
- [2026-06-26] — Confirmed fan-out design principles for the 90-node topology:
 - Few core databases (4-5), each read by 8-15 dependent services — drives PageRank/eigenvector for data-tier nodes
 - 1-2 true betweenness chokepoints that sit *between* tiers without storing data themselves (e.g. an auth gateway) — distinguishes betweenness from PageRank/eigenvector
 - 25-30% of nodes as a long tail with in/out-degree of 1-2 (single-purpose tools, batch jobs) — anchors sparsity, makes high-centrality nodes stand out by contrast
 - 2-3 services with partial (not full) redundancy — avoids a naive “everything critical has a backup” reading
- [2026-06-26] — Confirmed six deliberately planted structural risks, sized for divergence across the three centrality metrics:

#	Node	Risk	Dominant metric(s)
1	Auth Gateway	No redundant path; all authenticated traffic flows through it	Betweenness high; PageRank moderate
2	Core Ledger DB	Single primary, read by nearly every financial service, no read replica	Eigenvector + PageRank high; betweenness lower (endpoint, not pass-through)
3	Legacy Billing Service	Predates microservices migration; payment paths still route through it for reconciliation	Betweenness high
4	Shared Message Queue	12+ services publish/subscribe through one unpartitioned cluster	Betweenness + eigenvector both high (metrics agree)
5	Third-Party KYC/AML Vendor API	No circuit breaker or cached fallback; onboarding and compliance both block on it	Low on all three centrality metrics; high blast radius on simulation — the deliberate counterexample proving centrality alone is insufficient
6	Secrets Vault	Every service authenticates to every data store through one vault instance	PageRank high (transitive dependency); betweenness moderate

Risk #5 is the methodological anchor for Section 3.7 (Blast Radius Validation) — it is the node that demonstrates why blast radius simulation is required *in addition to* centrality ranking, directly supporting the “On Methodological Validity” argument pattern established in the AD project.

13. Change Log

- [2026-06-26] — Initial project setup; INSTRUCTIONS.md created following project outline discussion